Optimización Eco-Eficiente de Rutas para Transporte de Pulpa de Café

Contents

1 Introducción **2**
- 1.1 Propósito . . . 2
- 1.2 Ámbito del sistema . . . 2
- 1.3 Personal Involucrado . . . 2
- 1.4 Definiciones, Acrónimos y Abreviaturas . . . 3
- 1.5 Referencias . . . 4

2 Descripción general del documento **4**
- 2.1 Perspectiva del producto . . . 4
- 2.2 Características de los usuarios . . . 4
- 2.3 Restricciones . . . 5
- 2.4 Requisitos futuros . . . 5

3 Requisitos específicos **5**

4 Apéndices **9**
- 4.1 Gestión y optimización de memoria en el motor V8 . . . 9
- 4.2 La pila de llamadas (Call Stack): velocidad y control de flujo . . . 10
- 4.3 El montón de memoria (Memory Heap): almacenamiento dinámico . . . 10
- 4.4 Comportamiento operativo: copias reales frente a referencias compartidas . . . 11
- 4.5 Desafíos de rendimiento: latencia y fugas de memoria . . . 11
- 4.6 Relación con los requerimientos del sistema . . . 11

5 Bibliografia **11**

1 Introducción

1.1 Propósito

El propósito de este documento de Especificación de Requisitos Software (ERS) es definir de forma clara y verificable los requisitos funcionales y no funcionales del sistema de **Optimización Eco-Eficiente de Rutas para el Transporte de Pulpa de Café**. Está dirigido al equipo de desarrollo, a los docentes o evaluadores del proyecto, y a los usuarios finales (productores cafeteros, cooperativas y responsables de logística), quienes lo utilizarán como referencia para el diseño, la implementación y la validación del sistema.

1.2 Ámbito del sistema

En esta subsección:

- Se dará el nombre de **EcoRutaCafé** al futuro sistema (nombre provisional).
- Se explicará lo que el sistema hará: calcular rutas óptimas para recolectar y transportar pulpa de café desde las fincas y beneficiaderos hasta los puntos de aprovechamiento (compostaje, biogás, abono), minimizando distancia, tiempo, consumo de combustible y emisiones de CO_2. Lo que no hará: no gestionará la facturación, la contabilidad ni la producción del café, ni controlará directamente los vehículos.
- Se describirán los beneficios, objetivos y metas: reducir costos operativos de transporte, disminuir la huella de carbono, evitar la contaminación por mala disposición de la pulpa y promover su aprovechamiento como residuo valioso dentro de la economía circular.
- Se referenciarán los documentos de nivel superior (plan del proyecto, normativa ambiental aplicable y, si existe, el documento de requisitos globales del sistema), con los que este ERS debe mantener consistencia.

1.3 Personal Involucrado

Nombre	Irvin Alexey Armijos Guerra
Rol	Programador
Categoria Profesional	Estudiante
Resposabilidad	Diseño y Programacion
Informacion de contacto	irvin.armijos@unl.edu.ec

Nombre	Bayron Alexander Coronoel Sarango
Rol	Programador
Categoria Profesional	Estudiante
Resposabilidad	Diseño y Programacion
Informacion de contacto	byron.coronel@unl.edu.ec

Nombre	Cesar Luis Lojan Campoverde
Rol	Programador
Categoria Profesional	Estudiante
Resposabilidad	Diseño y Programacion
Informacion de contacto	cesar.lojan@unl.edu.ec

Nombre	Lucas Matias Romero Pacheco
Rol	Programador
Categoria Profesional	Estudiante
Resposabilidad	Diseño y Programacion
Informacion de contacto	lucas.m.romero@unl.edu.ec

Nombre	Gabriel Alejandro Suarez Poma
Rol	Programador
Categoria Profesional	Estudiante
Resposabilidad	Diseño y Programacion
Informacion de contacto	gabriel.suarez@unl.edu.ec

1.4 Definiciones, Acrónimos y Abreviaturas

En esta subsección se definirán todos los términos, acrónimos y abreviaturas utilizadas en la ERS.

- **ERS:** Especificación de Requisitos Software.
- **RF:** Requerimiento Funcional.
- **RNF:** Requerimiento No Funcional.
- **EcoRutaCafé:** Nombre provisional del sistema de optimización eco-eficiente de rutas para el transporte de pulpa de café.
- **Grafo dirigido:** Grafo en el que las conexiones entre los nodos tienen un sentido definido.
- **Nodo:** Elemento que forma parte del grafo. En el sistema, cada finca registrada se convertirá en un nodo de la red vial.

- **Arista:** Conexión entre dos nodos del grafo que representa un tramo de la red vial.
- **Peso:** Valor asignado a una arista del grafo, calculado a partir de la distancia, la pendiente y el coeficiente de emisión vehicular.
- **API:** Interfaz utilizada por el backend para permitir que el frontend consuma las funciones del sistema.
- **Backend:** Parte del sistema que contiene el motor del grafo y expone la API.
- **Frontend:** Parte del sistema que consume la API y no contiene la lógica de cálculo de rutas.
- **Altimetría:** Información relacionada con la elevación del terreno utilizada para el procesamiento de las rutas.

1.5 Referencias

Título del Documento	Referencia
Standard IEEE 830 – 1998	IEEE

2 Descripción general del documento

2.1 Perspectiva del producto

EcoRutaCafe será un producto diseñado para trabajar en entornos web, lo que permitira su utilización de forma rápida y sencilla desde cualquier dispositivo. El sistema es independiente y se apoyará en mapas interactivos para mostrar las rutas, con el fin de apoyar la toma de decisiones en el transporte de pulpa de café.

2.2 Características de los usuarios

Usuario	Descripción	Nivel educativo	Experiencia técnica
Administrador	Registra las fincas y carga los datos estáticos de la red vial (RF-01, RF-02).	Medio o superior	Media: maneja computadora y formularios web
Responsable de logística	Solicita rutas y consulta los reportes de emisiones (RF-04, RF-05, RF-06).	Medio o superior	Media: usa herramientas de planificación
Productor cafetero o cooperativa	Consulta las rutas de recolección de su pulpa de café.	Básico a medio	Baja: uso básico de aplicaciones web

En general, los usuarios no son especialistas en informática ni en optimización de grafos. Por ello, la interfaz debe ser clara y sencilla, y no exigir conocimientos técnicos para solicitar una ruta o interpretar un reporte.

2.3 Restricciones

- **Lenguajes y tecnologías:** el *backend* debe implementarse en Node.js y el *frontend* en Vue.js (RNF-02).
- **Arquitectura e interfaces:** el *frontend* solo puede comunicarse con el sistema a través de la API del *backend* y no debe contener lógica de cálculo de rutas (RNF-02).
- **Limitaciones del hardware:** el motor del grafo debe operar dentro de un límite de memoria heap (RNF-04) y calcular cada ruta en un máximo de 3 segundos (RNF-01).
- **Datos de entrada:** la altimetría debe tener una resolución de al menos 30 m (RNF-03), y los datos de la red vial se cargan una sola vez como información de solo lectura (RF-02).
- **Interfaces con otras aplicaciones:** el sistema no se integra con sistemas de facturación, contabilidad ni control de vehículos.

2.4 Requisitos futuros

ID	Nombre	Descripción	Extiende a
RF-F01	Ruta de recolección con varias fincas	El sistema deberá calcular una ruta que recoja la pulpa de varias fincas en un solo recorrido, en lugar de un único par origen–destino.	RF-04, RF-06
RF-F02	Actualización de datos viales	El sistema deberá permitir actualizar los datos de la red vial sin recargarla por completo (por ejemplo, tramos cerrados o en mal estado).	RF-02
RNF-F01	Escalabilidad de la red vial	El motor del grafo deberá mantener el consumo de memoria y el tiempo de cálculo en límites aceptables al ampliar la zona más allá de Loja.	RNF-01, RNF-04

3 Requisitos específicos

Requerimientos Funcionales

Identificación del requerimiento:	RF-01
Nombre del requerimiento:	Gestión de fincas cafeteras
Características:	El sistema deberá permitir registrar, editar y eliminar los datos de las distintas fincas cafeteras.
Descripción del requerimiento:	El administrador podrá crear, modificar y eliminar fincas, almacenando datos como nombre, ubicación (coordenadas) y cantidad de pulpa de café a recolectar. Cada finca registrada se convertirá en un nodo de la red vial.
Requerimiento NO funcional:	• RNF-02
Prioridad del requerimiento: Media	

Identificación del requerimiento:	RF-02
Nombre del requerimiento:	Ingreso de datos estáticos de la red vial
Características:	El sistema deberá permitir agregar los datos estáticos de la red vial.
Descripción del requerimiento:	El sistema recibirá y almacenará los tramos viales con su origen, destino, distancia, pendiente y sentido, junto con los datos de elevación del terreno. Estos datos se cargarán una sola vez y se mantendrán como información de solo lectura durante las consultas.
Requerimiento NO funcional:	• RNF-02 • RNF-03
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF-03
Nombre del requerimiento:	Construcción del grafo dirigido
Características:	El sistema deberá construir un grafo dirigido a partir de los datos ingresados.
Descripción del requerimiento:	A partir de las fincas y los tramos viales, el sistema generará un grafo dirigido valorado en el que cada arista tiene un peso calculado con distancia, pendiente y coeficiente de emisión vehicular.
Requerimiento NO funcional:	• RNF-01 • RNF-03
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF-04
Nombre del requerimiento:	Validación de origen, destino y conectividad
Características:	El sistema deberá validar que el origen y el destino existan y estén conectados, e informar si no existe una ruta.
Descripción del requerimiento:	Antes de calcular, el sistema comprobará que ambos nodos estén en el grafo y que exista al menos un camino entre ellos. Si no existe, mostrará al usuario un mensaje claro indicando que no hay ruta posible.
Requerimiento NO funcional:	• RNF-01 • RNF-02
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF-05
Nombre del requerimiento:	Reporte comparativo de emisiones
Características:	El sistema generará un reporte comparativo con distancias, emisiones y porcentaje de reducción de emisiones.
Descripción del requerimiento:	Para un mismo origen y destino, el reporte comparará la ruta convencional (mínima distancia) con la ruta eco-optimizada, mostrando distancia (km), emisiones (kg de CO_2) y el porcentaje de reducción de emisiones obtenido.
Requerimiento NO funcional:	• RNF-02 • RNF-03
Prioridad del requerimiento: Alta	

Requerimientos No Funcionales

Identificación del requerimiento:	RNF-01
Nombre del requerimiento:	Tiempo de cálculo de ruta
Descripción del requerimiento:	El cálculo de una ruta deberá tardar como máximo 3 segundos.
Forma de verificación:	Medición del tiempo de ejecución con `performance.now()` sobre el grafo de la red vial de Loja.
Requerimiento funcional:	• RF-03 • RF-04
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RNF-02
Nombre del requerimiento:	Separación entre backend y frontend
Descripción del requerimiento:	El sistema deberá implementarse separando el backend del frontend.
Forma de verificación:	El backend (Node.js) expondrá una API que contiene el motor del grafo, y el frontend (Vue.js) solo consumirá esa API. La verificación se hace por revisión de la arquitectura: el frontend no debe contener lógica de cálculo de rutas.
Requerimiento funcional:	• RF-01 • RF-02 • RF-04 • RF-05
Prioridad del requerimiento: Media	

Identificación del requerimiento:	RNF-03
Nombre del requerimiento:	Procesamiento de altimetría e inclinación
Descripción del requerimiento:	El sistema deberá procesar datos de altimetría e inclinación del terreno con una resolución de al menos 30 metros (datos estándar de elevación) para las rutas intercantonales lojanas.
Forma de verificación:	Carga de un modelo digital de elevación de resolución igual o mejor a 30 m sobre la zona de Loja y comprobación de que la pendiente de cada tramo se calcula a partir de esos datos.
Requerimiento funcional:	• RF-02 • RF-03 • RF-05
Prioridad del requerimiento: Alta	

4 Apéndices

4.1 Gestión y optimización de memoria en el motor V8

El motor V8, desarrollado por Google, es el entorno de ejecución de alto rendimiento responsable de procesar y compilar código JavaScript en navegadores como Chrome y en entornos

de servidor como Node.js. Para que las aplicaciones se ejecuten de manera fluida, V8 implementa un sistema de gestión de memoria que divide los recursos del sistema operativo en dos grandes áreas de trabajo, cada una optimizada para un tipo específico de tarea y de estructura de datos: la pila de llamadas y el montón de memoria [4].

4.2 La pila de llamadas (Call Stack): velocidad y control de flujo

La pila de llamadas opera como el centro de control inmediato del motor. Es una estructura de memoria estática, extremadamente rápida y rígidamente organizada, cuyo propósito principal es rastrear el punto exacto de ejecución del programa y gestionar el contexto inmediato [5].

Mecanismo de funcionamiento. Opera bajo el principio LIFO (*Last-In, First-Out*, es decir, último en entrar, primero en salir). Cada vez que la aplicación ejecuta una función, V8 crea un "marco de ejecución" y lo apila en la cima. Al finalizar dicha función, el marco se retira inmediatamente, devolviendo el control a la función que estaba debajo.

Datos alojados. Debido a su diseño enfocado en la latencia ultrabaja, la pila almacena principalmente datos cuyo tamaño físico es fijo y conocido, como valores simples (enteros pequeños y booleanos) y los punteros: direcciones de memoria de 64 bits que actúan como "mapas" hacia la información guardada en el montón.

Ciclo de limpieza. Su arquitectura hace que la gestión del espacio sea totalmente automática. No requiere procesos en segundo plano: el espacio se libera al instante en cuanto el marco de la función es retirado de la pila. Sin embargo, su capacidad total es limitada. Una acumulación excesiva de funciones no resueltas (como en una recursividad infinita) provoca el colapso conocido como *Stack Overflow* (desbordamiento de pila).

4.3 El montón de memoria (Memory Heap): almacenamiento dinámico

Mientras la pila prioriza el orden y la inmediatez, el montón prioriza la capacidad y la flexibilidad estructural. Es una región amplia y menos estructurada, diseñada para alojar información cuyo tamaño es variable o impredecible [6].

Datos alojados. En esta región residen las estructuras de datos complejas, conocidas genéricamente como "tipos de referencia": objetos, arreglos, funciones y clases instanciadas.

Mecanismo de acceso. El acceso a los datos en el montón es indirecto. Cuando el programa necesita procesar un objeto, V8 debe ejecutar dos pasos: primero, buscar en la pila el puntero correspondiente; segundo, realizar la desreferenciación, viajando a la ubicación exacta en el montón para leer o alterar los datos.

Ciclo de limpieza. El montón no se vacía automáticamente. Dado que las estructuras complejas pueden sobrevivir a la función que las creó, V8 utiliza un subsistema interno llamado *Garbage Collector* (recolector de basura). Este escanea el montón mediante algoritmos como *Mark-and-Sweep* (marcar y barrer) para identificar y destruir los objetos que el programa ya no necesita.

4.4 Comportamiento operativo: copias reales frente a referencias compartidas

Gran parte de los errores lógicos en el desarrollo de software ocurren por no comprender cómo interactúan estas dos áreas al asignar variables.

Asignación por valor (datos simples). Cuando se declara una variable con un valor primitivo (por ejemplo, `let a = 1`) y se copia a otra (`let b = a`), V8 reserva dos celdas independientes. Modificar la variable `b` altera únicamente su propia celda y mantiene intacto el valor de `a`.

Asignación por referencia (datos complejos). Cuando se crea un objeto, la estructura se guarda en el montón y en la pila solo se anota su dirección. Si esta variable se copia, V8 **no duplica el objeto**: simplemente copia el puntero. Como resultado, ambas variables apuntan al mismo bloque de datos, y una mutación realizada desde una de ellas se refleja en la otra.

4.5 Desafíos de rendimiento: latencia y fugas de memoria

Una gestión deficiente de estas mecánicas somete al motor a cargas de trabajo innecesarias que degradan la experiencia del usuario final.

Sobrecarga del recolector de basura (pausas *Stop-The-World*). Para que el algoritmo *Mark-and-Sweep* elimine objetos del montón de forma segura, necesita detener momentáneamente el hilo de ejecución principal. Si una aplicación crea y destruye miles de objetos temporales por segundo, el recolector se ejecuta de forma continua. Estas pausas repetitivas bloquean el renderizado de la interfaz y provocan que la aplicación se congele o responda con lentitud (fenómeno conocido como *jank*).

Fugas de memoria (*memory leaks*). Una fuga de memoria es una acumulación de datos que ya no se usan. Ocurre cuando un objeto en el montón terminó su utilidad, pero por un error de diseño (un temporizador no detenido, variables globales olvidadas, elementos gráficos ocultos pero no destruidos) el programa mantiene un puntero activo hacia él. Al detectar esa referencia viva, el recolector asume que el objeto sigue siendo necesario y no lo elimina. Con el tiempo, esta acumulación consume la memoria dinámica disponible y puede terminar en el cierre forzoso de la aplicación [4].

4.6 Relación con los requerimientos del sistema

Estos conceptos justifican los requerimientos de memoria del sistema. Un grafo vial con un objeto por arista implicaría millones de objetos en el montón, con más referencias y más trabajo para el recolector de basura. Por ello, el RNF-05 establece almacenar los atributos numéricos en arreglos tipados, y el RNF-04 acota el consumo de memoria del motor del grafo.

5 Bibliografia

[1] Node.js Learn, "Understanding and Tuning Memory | Node.js Learn," nodejs.org, Apr. 10, 2025. [Online]. Available: https://nodejs.org/learn/diagnostics/memory/understanding-

and-tuning-memory. [Accessed: Oct. 07, 2026] [cite: 1.1.1].

[2] A. Estructura de Datos, "MEMORIA ESTATICA Y DINAMICA - ESTRUCTURA DE DATOS," adrian-estructuradedatos.blogspot.com, Apr. 03, 2011. [Online]. Available: http://adrian-estructuradedatos.blogspot.com/2011/04/memoria-estatica-y-dinamica.html. [Accessed: Oct. 07, 2026] [cite: 1.2.1].

[3] F. Estrada, "Gestión de Memoria Dinámica y Estructuras de Datos en C++," medium.com, Feb. 06, 2025. [Online]. Available: https://medium.com/@Mellamoluis16/gesti